

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO
DEPARTAMENTO DE COMPUTAÇÃO

Marcelo da Silva Zanata – 16831070

Relatório

Busca e Organização de Dados

SÃO CARLOS – SP
2026

1. Análise da Complexidade Assintótica das Operações

A tabela abaixo sintetiza a complexidade assintótica de tempo no pior caso (*O)* das operações fundamentais implementadas para cada uma das 8 estruturas de dados, considerando *N* como a quantidade total de registros:

Tabela 1: Complexidade Assintótica Pior Caso das Operações por Estrutura de Dados

Estrutura	Inserção	Busca / Percurso	Remoção	Observações Teóricas
pilha	O(1)	O(N)	O(1)	Disciplina LIFO no topo. Inserção e remoção imediatas por ponteiro. Percurso exige varredura linear do topo à base.
fila	O(1)	O(N)	O(1)	Disciplina FIFO. Ponteiros para inicio e fim garantem custo constante tanto para inserção no fim quanto para desenfileirar do início.
lista sequencial	O(1)* amort.	O(N)	O(1) no fim	Array contíguo dinâmico com política de dobra de capacidade geométrica. Acesso por índice direto em O(1).

Estrutura	Inserção	Busca / Percurso	Remoção	Observações Teóricas
lista encadeada	O(1) com calda	O(N)	O(N)	Alocação dinâmica sob demanda nó a nó. Mantendo ponteiro fim, a inserção é O(1). A remoção arbitrária requer localizar o nó anterior (O(N)).
lista com cabeça	O(1) com calda	O(N)	O(N)	Utiliza nó sentinela real estático. Elimina checagem de ponteiro nulo para o primeiro nó, padronizando os invariantes de encadeamento.
lista ordenada	O(N)	O(N)	O(N)	A ordem é estritamente mantida na inserção. Cada inserção localiza a posição correta (O(N)), totalizando $O(N^2)$ para N itens.
lista generalizada	$O(G_1 + G_2)$	O(N)	O(N)	Hierarquia em 2 níveis (Grupo → Subgrupo → Item). Inserção busca ou cria o nó de grupo e subgrupo antes de encadear o registro.

Estrutura	Inserção	Busca / Percurso	Remoção	Observações Teóricas
lista cruzada	$O(E_1 + E_2)$	O(N)	O(N)	Estrutura ortogonal multi-eixo. Um único nó físico é encadeado em duas listas ortogonais simultâneas sem duplicar a carga de dados.

2. Metodologia Experimental

Os experimentos foram conduzidos em ambiente Linux (CachyOS), compilado via GCC com otimização -O2, utilizando medição em microssegundos via clock_gettime(CLOCK_MONOTONIC). Cada configuração foi repetida **3 vezes**, sendo apresentada a média aritmética dos tempos observados.

Configurações avaliadas:

- **Variação do tamanho de entrada:** N ∈ {1.000, 5.000, 10.000, 25.000, 50.000, toda a base}.
- **Variação do parâmetro k:** k ∈ {10, 50, 100, 500, 1000, todos}.
- **Par de estruturas canônico:** Base = lista_enc, Resultado = lista_ord (ordenada por similaridade decrescente).

3. Quadro de Avaliação Experimental de Tempos (Médias em milissegundos)

Tabela 2: Medição dos Tempos de Carga, Busca, Inserção e Escrita

N (Registros)	Base	Resultado	k	Load (ms)	Busca (ms)	Inserção Res. (ms)	Saída (ms)	Total (ms)
1.000	**lista_enc**	**lista_ord**	**10**	4.756	0.041	4.984	0.230	**20.01**
5.000	**lista_enc**	**lista_ord**	**50**	24.552	0.279	191.055	0.637	**216.52**
10.000	**lista_enc**	**lista_ord**	**100**	46.812	0.643	1002.330	0.886	**1050.67**
25.000	**lista_enc**	**lista_ord**	**500**	99.330	2.529	88.40	2.623	**192.88**

N (Registros)	Base	Resultado	k	Load (ms)	Busca (ms)	Inserção Res. (ms)	Saída (ms)	Total (ms)
50.000	**lista_enc**	**lista_ord**	**1000**	184.178	6.174	36574.113	5.581	**36770.05**
Toda base	**lista_seq**	**fila**	**todos**	326.720	2.800	5.774	272.672	**607.97**

4. Discussão Comparativa dos Resultados

4.1. Tempo de Carga (LOAD_MS)

Apresenta crescimento estritamente linear O(N) em relação ao número de linhas lidas. O uso de parsing CSV em memória com alocação otimizada e inferência de tipos dinâmica garantiu vazão superior a 200.000 registros por segundo. A lista_seq e a lista_enc com ponteiro para calda apresentaram os menores tempos de carga, pois inserem em O(1) no fim.

4.2. Tempo de Busca (SEARCH_MS)

A avaliação dos critérios FILTER e cálculo de similaridade ponderada SIM s(x, q, t) = max(0, 1 - |x - q| / t) ocorre em O(1) por registro. Assim, a busca na base cresce proporcionalmente ao tamanho da base: O(N). Para N = 50.000, o tempo de busca total permaneceu em torno de apenas 6,174 ms.

4.3. Tempo de Inserção nos Resultados (RESULT_INSERT_MS)

Quando --result é configurado como lista_ord, cada elemento aceito é inserido em sua posição exata de ordenação (O(M), onde M é o número de registros filtrados). O custo acumulado dessa fase é de $O(M^2)$, o que explica o crescimento quadrático observado na tabela experimental quando a quantidade de registros aceitos cresce com N. Em contrapartida, quando --result é configurado como fila ou pilha, a inserção é O(1) por item, totalizando tempo praticamente desprezível (5.774ms para toda a base).

4.4. Tempo de Escrita (WRITE_MS)

Cresce linearmente em função de k (O(k)). O delimitador e o cabeçalho original são rigorosamente preservados, sem inclusão de colunas auxiliares no CSV final, minimizando I/O de disco.